



Modelação e Simulação de Sistemas Naturais

ISEL - DEETC - LEIM

Projeto Final

Docente: Arnaldo Joaquim Abrantes

Trabalho realizado por:

Martim Ramos nº 51736
Guilherme Graça nº 51827

Licenciatura em Engenharia Informática e Multimédia
Semestre de Inverno 2024/2025

Turma: LEIM32D

19 de Janeiro de 2025

Índice

Índice.....	1
1. Introdução do Projeto.....	2
2. Desenvolvimento do Ecossistema.....	3
2.1 Alterações face ao código fornecido.....	4
3. Resultados obtidos.....	18
4. Discussão de Resultados.....	18
5. Conclusão.....	19
6. Bibliografia.....	19

1. Introdução do Projeto

No âmbito da disciplina, o projeto final desenvolvido consiste na implementação de um ecossistema animal entre presas e predadores.

A fim de recriar um exemplo que simula o ciclo de vida animal, criámos um ecossistema que recebe por base um terreno terrestre e aquático. Existem espaços férteis que geram alimento (cerejas) às presas (ratos) bem como espaços que dificultam a vida a estas (zonas desertas e obstáculos presentes, como é o caso de lagos existentes no nosso ecossistema) fazendo com que tenham que lutar pela sua sobrevivência. A presença de predadores (raposas) torna o nosso ecossistema mais interessante, pois estes alimentam-se das nossas presas.

No nosso ecossistema acaba por existir um certo equilíbrio na população de ambas as espécies pois está presente o ciclo de vida de cada uma no nosso ecossistema, ou seja, ambas nascem inicialmente, reproduzem-se e morrem.

As presas e os predadores ao reproduzirem-se podem gerar descendentes com mutações. Uma vez que, estes herdam os genes dos pais, ao longo do percurso de vida de cada animal, acabam por sobreviver aqueles que são mais aptos às condições do ambiente pois os recursos presentes nele são limitados.

Com a intenção de dar asas a este projeto, desenvolvemos um simulador interativo com o utilizador que tenta representar o nosso ecossistema. Este projeto foi desenvolvido em ambiente Java com recurso à biblioteca Processing.

2. Desenvolvimento do Ecossistema

Nesta fase do projeto, abordar-se-á todo o processo realizado ao longo do processo de implementação do mesmo.

Como ponto de partida os vídeos fornecidos pelo professor, o projeto está estruturado com a seguinte forma:

- package **agentes_autonomos**: Responsável por gerir o comportamento dos boids (presas e predadores).
- package **ca**: Responsável por criar as células que irão constituir o autômato celular (terreno) bem como as regras da maioria.
- package **ecosystem**: Responsável definir tudo o que é englobado no nosso ecossistema, como é o caso das bases que constituem os nossos animais (a sua energia, a energia que recolhem do seu alimento, a sua massa, etc.), as suas populações, as condições presentes no terreno e por fim criar as nossas presas e os nossos predadores.
- package **physics**: Responsável por dar sustento à criação dos boids pois estes são também constituídos por um corpo.
- package **tools**: Package responsável por armazenar ferramentas importantes que servem de base para a construção do nosso projeto, como por exemplo, a classe Subplot.
- package **setup**: Responsável por correr o nosso programa.
- package **Imagens**: Responsável por armazenar imagens utilizadas no nosso projeto.

2.1 Alterações face ao código fornecido

- Criação da classe Predador:

Esta classe tem por base a estrutura da classe Prey mas com novos atributos, imagens diferentes e é utilizado o método eatPrey(Poulation population) invés do método eat(Terrain terrain). Este novo método foi adicionado à interface IAnimal e à classe Animal que implementa esta interface.

Novos atributos do predador (WorldConstants):

```
public final static float PREDATOR_SIZE = .2f; 1 usage
public final static float PREDATOR_MASS = 1f; 1 usage
public final static int INI_PREDATOR_POPULATION = 15; 1 usage
public final static float INI_PREDATOR_ENERGY = 30f; 3 usages
public final static float PREDATOR_ENERGY_TO_REPRODUCE = 70f; 1 usage
public static int[] PREDATOR_COLOR = {255,0,0}; 3 usages
```

Método eatPrey(Poulation population):

Este método permite com que o predador localize as presas mais próximas, fazendo recurso de um método auxiliar adicionado na classe population que retorna as preys, e consuma-as. É verificada a distância entre o predador e as presas e, ao encontrar uma presa dentro do alcance, transfere a energia da presa para o predador e por fim remove a presa utilizando o método kill da classe Animal.

```
@Override 1 usage
public void eatPrey(Population population) {
    for (Animal prey : population.getPreys()) {
        if (PVector.dist(this.pos, prey.getPos()) < this.radius * 2 ) {
            energy += prey.energy;
            prey.kill();
            break;
        }
    }
}
```

Interface IAnimal com a adição do método eatPrey():

```
public interface IAnimal { 1 usage 3 implementations
    public Animal reproduce(boolean mutate); 1 usage 3 implementations
    public void eat(Terrain terrain); 1 usage 2 implementations
    public void eatPrey(Population populatuion); 1 usage 2 implem
    public void energy_consumption(float dt, Terrain terrain);
    public boolean die(); 1 usage 1 implementation
}
```

Classe Animal com o método eatPrey() na sua base:

```
@Override 1 usage 1 override
public void eatPrey(Population populatuion) {

}
```

Método getPreys():

```
public List<Animal> getPreys() { 2 usages
    List<Animal> preys = new ArrayList<>();
    for (Animal animal : allAnimals) {
        if (animal instanceof Prey) {
            preys.add(animal);
        }
    }
    return preys;
}
```

- Comportamento do predador de seguir a presa:

Classe Eye:

Alteração no método look():

O método sofreu alterações com o objetivo de escolher o alvo mais próximo. Se este alvo estiver dentro do alcance e do ângulo de visão é adicionado à lista farSight. De seguida é efetuado o cálculo da distância entre o agente (me.getPos()) e o corpo b com recurso a PVector.dist(). O alvo mais próximo é atualizado se a distância for menor que minDistance. Se isto acontecer o corpo b passa a ser o mais próximo (alvo) e minDistance é atualizado. Foi utilizado o comportamento Pursuit criado anteriormente no decorrer da disciplina pelo professor.

```
public void look(){ 2 usages
    farSight = new ArrayList<Body>();
    nearSight = new ArrayList<Body>();
    Body closestPrey = null;
    float minDistance = Float.MAX_VALUE; //valor maximo de um float

    for (Body b : allTrackingBodies){
        if (farSight(b.getPos())){
            farSight.add(b);
            float distance = PVector.dist(me.getPos(), b.getPos());
            if (distance < minDistance) {
                minDistance = distance;
                closestPrey = b;
            }
        }
        if (nearSight(b.getPos())){
            nearSight.add(b);
        }
    }

    if (closestPrey != null) {
        target = closestPrey;
    }
}
```

- Alterações na classe Boid:

Para que os filhos herdem o comportamento de Pursuit da mesma forma que as preys herdem o comportamento de AvoidObstacles, foi feita a seguinte alteração no método mutateBehaviors().

```
public void mutateBehaviors(){ 2 usages
    for (Behavior behavior : behaviors){
        if (behavior instanceof AvoidObstacle){
            behavior.weight += DNA.random(-0.5f,0.5f);
            behavior.weight = Math.max(0,behavior.weight); //acima de 0
        }
        if (behavior instanceof Pursuit){
            behavior.weight += DNA.random(-0.5f,0.5f);
            behavior.weight = Math.max(0,behavior.weight);
            behavior.weight = Math.min(4, behavior.weight); //abaixo de 4
        }
    }
    updateSumWeights();
}
```

O peso do comportamento Pursuit não passa de 4 pois notou-se que os predadores juntam-se no mesmo sítio se o weight for muito elevado por algum motivo.

Predadores são inicializados como as preys mas com o comportamento de Pursuit invés de Wander.

```
for(int i = 0; i < WorldConstants.INI_PREDATOR_POPULATION; i++){
    PVector pos = new PVector(parent.random((float) window[0], (float) window[1]),
        parent.random((float) window[2], (float) window[3]));
    int color = parent.color(
        WorldConstants.PREDATOR_COLOR[0],
        WorldConstants.PREDATOR_COLOR[1],
        WorldConstants.PREDATOR_COLOR[2]);
    //PVector() foi adicional por causa que boid contem a vel como parametro
    Animal a = new Predador(pos, WorldConstants.PREDATOR_MASS, WorldConstants.PREDATOR_SIZE, color, parent, plt);
    a.addBehavior(new Wander( weight: 1));
    a.addBehavior(new Pursuit( weight: 2));
    Eye eye = new Eye(a, this.getPreyBodies());
    a.setEye(eye);
    allAnimals.add(a);
}
```

Para poder utilizar o Pursuit, o olho da presa é inicializado com uma lista dos bodies das presas, para isso foi criado o método `getPreyBodies()`. Este método percorre todas as presas na lista de presas e cria um array de bodies baseado na posição de cada presa.

```
public List<Body> getPreyBodies() { 1 usage
    List<Body> preyBodies = new ArrayList<>();
    // Percorre todas as presas
    for (Animal prey : this.getPreys()) {
        // Cria um Body baseado na posição de cada presa
        Body b = new Body(prey.getPos());
        preyBodies.add(b);
    }
    return preyBodies;
}
```

Visualização das presas e dos predadores no canvas:

```
private PImage boidImage1;
private PImage boidImage2;
```

```
public void setPreyImage(PImage img) { this.boidImage1 = img; }
```

```
public void setPredatorImage(PImage img) { this.boidImage2 = img; }
```

```
@Override 1 usage
public void display(PApplet p, SubPlot plt) {
    float[] pp = plt.getPixelCoord(pos.x, pos.y);

    if (boidImage1 != null) {
        p.pushMatrix();
        p.translate(pp[0], pp[1]);
        //p.rotate(-vel.heading()); // Ajusta a rotação para a direção do Boid
        p.imageMode(PApplet.CENTER);
        p.image(boidImage1, a: 0, b: 0, c: 50, d: 50);
        p.popMatrix();
    }

    if (boidImage2 != null) {
        p.pushMatrix();
        p.translate(pp[0], pp[1]);
        //p.rotate(-vel.heading()); // Ajusta a rotação para a direção do Boid
        p.imageMode(PApplet.CENTER);
        p.image(boidImage2, a: 0, b: 0, c: 50, d: 50); // Desenha a imagem na posição rotacionada
        p.popMatrix();
    }
}
```

boidImage1 - imagem das presas
boidImage2 - imagem dos predadores

Inicialização da imagem e chamada do método setPreyImage (imagem das presas):

```
// Presas que começam inicialmente
public Prey(PVector pos, float mass, float radius, int color, PApplet parent, SubPlot plt) { 1 usage
    super(pos, mass, radius, color, parent, plt);
    this.parent = parent;
    this.plt = plt;
    this.preyImage = parent.loadImage( filename: "Imagens/mouse.png");
    setPreyImage(preyImage);
    energy = WorldConstants.INI_PREY_ENERGY;
}

// Presas filho que recebem os genes das presas pai
public Prey(Prey prey, boolean mutate, PApplet parent, SubPlot plt) { 1 usage
    super(prey, mutate, parent, plt);
    this.parent = parent;
    this.plt = plt;
    this.preyImage = parent.loadImage( filename: "Imagens/mouse.png");
    setPreyImage(preyImage);
    energy = WorldConstants.INI_PREY_ENERGY;
}
```

Inicialização da imagem e chamada do método setPredatorImage (imagem dos predadores):

```
public Predador(PVector pos, float mass, float radius, int color, PApplet parent, SubPlot plt) { 1 usage
    super(pos, mass, radius, color, parent, plt);
    this.parent = parent;
    this.plt = plt;
    this.predatorImage = parent.loadImage( filename: "Imagens/fox.png");
    setPredatorImage(predatorImage);
    energy = WorldConstants.INI_PREDATOR_ENERGY;
}

public Predador(Predador predador, boolean mutate, PApplet parent, SubPlot plt) { 1 usage
    super(predador, mutate, parent, plt);
    this.parent = parent;
    this.plt = plt;
    this.predatorImage = parent.loadImage( filename: "Imagens/fox.png");
    setPredatorImage(predatorImage);
    energy = WorldConstants.INI_PREDATOR_ENERGY;
}
```

- Alterações na classe Cell:

```
public class Cell { 17 usages 2 inheritors
    private int row, col; 2 usages
    protected int state; 9 usages
    private Cell[] neighbors; 3 usages
    protected CellularAutomata ca; 15 usages
    private int color; no usages

    private static PImage obstacleImage; 3 usages
    private static PImage foodImage; 3 usages

    public static void loadImages(PApplet p) { 1 usage
        obstacleImage = p.loadImage( filename: "Imagens/sea-waves.png");
        foodImage = p.loadImage( filename: "Imagens/cherries.png");
    }

    public void display(PApplet p){ 1 usage
        p.pushStyle();
        p.noStroke();
        p.fill(ca.getStateColors()[state]);
        float x = ca.xmin + col * ca.cellWidth;
        float y = ca.ymin + row * ca.cellHeight;
        p.rect(x, y, ca.cellWidth, ca.cellHeight);

        float centerX = x + ca.cellWidth / 2;
        float centerY = y + ca.cellHeight / 2;

        if (state == WorldConstants.PatchType.OBSTACLE.ordinal() && obstacleImage != null) {
            p.imageMode(PApplet.CENTER);
            p.image(obstacleImage, centerX, centerY, ca.cellWidth, ca.cellHeight);
        } else if (state == WorldConstants.PatchType.FOOD.ordinal() && foodImage != null) {
            p.imageMode(PApplet.CENTER);
            p.image(foodImage, centerX, centerY, ca.cellWidth, ca.cellHeight);
        }

        p.popStyle();
    }
}
```

- Alterações na classe Population:

As informações apresentadas na consola são iguais, no entanto para mostrar média dos pesos de cada comportamento das presas e dos predadores, o método

getMeanWeights foi substituído por 2 métodos diferentes (getMeanWeightsPrey() e getMeanWeightsPredators()).

```
public float[] getMeanWeightsPrey() { 2 usages
    float[] sums = new float[2];
    int count = 0;
    for (Animal a : allAnimals) {
        if (a instanceof Prey) {
            sums[0] += a.getBehaviors().get(0).getWeight(); // Wander
            sums[1] += a.getBehaviors().get(1).getWeight(); // AvoidObstacle
            count++;
        }
    }
    if (count > 0) {
        sums[0] /= count;
        sums[1] /= count;
    }
    return sums;
}
```

```
public float[] getMeanWeightsPredators() { 2 usages
    float[] sums = new float[2];
    int count = 0;
    for (Animal a : allAnimals) {
        if (a instanceof Predador) {
            sums[0] += a.getBehaviors().get(0).getWeight(); // Wander
            sums[1] += a.getBehaviors().get(1).getWeight(); // Pursuit
            count++;
        }
    }
    if (count > 0) {
        sums[0] /= count;
        sums[1] /= count;
    }
    return sums;
}
```

Métodos usados na classe EcoSystemApp:

```
System.out.println("PRESAS: "+ "meanWeightWander = " + population.getMeanWeightsPrey()[0] +  
    " meanWeightPursuit = " + population.getMeanWeightsPrey()[1]);  
System.out.println("PREDADORES: "+ "meanWeightWander = " + population.getMeanWeightsPredators()[0] +  
    " meanWeightPursuit = " + population.getMeanWeightsPredators()[1]);
```

- Barra de controlo para modificar o terreno:

Classe Button:

Responsável por criar um botão gráfico de interação com o utilizador.

```
public class Button { 8 usages  
    private float x, y, width, height; 6 usages  
    private int state; 2 usages  
    private int color; 2 usages  
    private boolean selected; 3 usages  
    private String label; 2 usages  
  
    public Button(float x, float y, float width, float height, int state, int color, String label) {  
        this.x = x;  
        this.y = y;  
        this.width = width;  
        this.height = height;  
        this.state = state;  
        this.color = color;  
        this.selected = false;  
        this.label = label;  
    }  
  
    public void setPosition(float x, float y) { 1 usage  
        this.x = x;  
        this.y = y;  
    }  
  
    public void setSize(float width, float height) { 1 usage  
        this.width = width;  
        this.height = height;  
    }  
}
```

```

public void display(PApplet p) { 1 usage
    p.pushStyle();
    p.strokeWeight(selected ? 4 : 1); //se estiver selecionado, aumenta a borda do btn
    p.stroke( rgb: 0);
    p.fill(color);
    p.rect(x, y, width, height);
    p.fill( rgb: 0);
    p.textAlign(PApplet.CENTER, PApplet.CENTER);
    p.textSize(20);
    p.text(label, x: x + width / 2, y: y + height / 2); // Adiciona o texto no centro do botão
    p.popStyle();
}

public boolean isMouseOver(PApplet p) { 1 usage
    return p.mouseX > x && p.mouseX < x + width && p.mouseY > y && p.mouseY < y + height;
}

public int getState() { 1 usage
    return state;
}

public void setSelected(boolean selected) { 2 usages
    this.selected = selected;
}

```

Classe ButtonBar:

Responsável por criar, organizar e gerir uma barra de botões interativos que é exibida no canvas do processing.

```

package tools;

import processing.core.PApplet;
import java.util.ArrayList;
import java.util.List;

public class ButtonBar { 3 usages
    private List<Button> buttons; 8 usages
    private int selectedState; 3 usages
    private float margin = 10; // entre os botões e os limites da barra

    public ButtonBar() { 1 usage
        buttons = new ArrayList<>();
        selectedState = -1; //estado selecionado
    }

    public void addButton(Button button) { 1 usage
        buttons.add(button);
    }
}

```

```

public void display(PApplet p) { 1 usage
    //fundo da barra
    p.pushStyle();
    p.fill( rgb: 120); //cinzento
    p.strokeWeight(3);
    p.rect( a: 0, b: p.height - 70, p.width, d: 70); //70 é a altura da barra
    p.popStyle();

    //(largura - nr de margens laterais)/ nr de botões
    float buttonWidth = (p.width - (buttons.size() + 1) * margin) / buttons.size();
    float buttonHeight = 40;
    //p.height - 70 = y superior da barra
    //(70 - buttonHeight) / 2 = margin vertical
    float buttonY = p.height - 70 + (70 - buttonHeight) / 2; // Centraliza os botões verticalmente
    for (int i = 0; i < buttons.size(); i++) {
        Button button = buttons.get(i);
        //começa só com o espaço da margem e salta a largura de um botão mais a margem
        button.setPosition( x: margin + i * (buttonWidth + margin), buttonY);
        button.setSize(buttonWidth, buttonHeight);
        button.display(p);
    }
}

```

```

public void handleMousePressed(PApplet p) { 1 usage
    for (Button button : buttons) {
        if (button.isMouseOver(p)) {
            //guarda o estado do botão selecionado
            selectedState = button.getState();
            for (Button btn : buttons) {
                btn.setSelected(false); //deseleciona todos os botões
            }
            button.setSelected(true); //seleciona o botão clicado
            break;
        }
    }
}

```

```

public int getSelectedState() { 1 usage
    return selectedState;
}

```

- Alterações na classe EcosystemApp:

Implementação da barra:

Viewports diferentes para serem usados em 2 subplots.

```
//barra
private float[] viewport = {0f, 0.1f, 1f, 0.9f}; 1 usage
private float[] buttonBarViewport = {0f, 0f, 1f, 0.1f}; 1 usage
private SubPlot buttonBarPlt; 1 usage
private ButtonBar buttonBar; 5 usages

//info da consola
private float timer, updateGraphTime; 5 usages
private float intervalUpdate = 1; 1 usage

//ecossistema
private SubPlot plt; 4 usages
private Terrain terrain; 10 usages
private Population population; 10 usages

@Override 1 usage
public void setup(PApplet p) {
    plt = new SubPlot(WorldConstants.WINDOW, viewport, p.width, p.height);
    buttonBarPlt = new SubPlot(WorldConstants.WINDOW, buttonBarViewport, p.width, p.height);
    Cell.loadImages(p); // Carrega as imagens para as células
    initializeEcosystem(p);
    initializeButtonBar(p);
}

private void initializeButtonBar(PApplet p) { 1 usage
    buttonBar = new ButtonBar();
    int[] colors = terrain.getStateColors();
    String[] labels = {"Vazio", "Obstáculo", "Comida"};

    for (int i = 0; i < labels.length; i++) {
        //salta a cor do terreno fértil pq n está nos botões
        Button button = new Button(x: 0, y: 0, width: 0, height: 0, i, colors[i == 2 ? i+1:i], labels[i]);
        buttonBar.addButton(button);
    }
}
```

```

@Override 1 usage
public void mousePressed(PApplet p) {
    if (p.mouseY > p.height * 0.9) { //botões
        buttonBar.handleMousePressed(p);
    } else { //terreno
        Cell clickedCell = terrain.pixel2Cell(p.mouseX, p.mouseY);
        if (clickedCell != null) {
            int selectedState = buttonBar.getSelectedState();
            if (selectedState != -1) {
                clickedCell.setState(selectedState);
            }
        }
    }
}
}

```

Método keyPressed(): Quando se clica na tecla 'SPACE' o programa reinicia.

```

@Override 1 usage
public void keyPressed(PApplet p) {
    if (p.key == ' ') {
        initializeEcosystem(p);
    }
}
}

```

Foram usadas 5 iterações da regra da maioria para que os patches fiquem maiores para dar mais realismo, como se fossem biomas. Se fosse só uma ia haver patches muito pequenos.

```

private void initializeEcosystem(PApplet p) { 2 usages
    terrain = new Terrain(p, plt);
    terrain.setStateColors(getColors(p));
    terrain.initRandomCustom(WorldConstants.PATCH_TYPE_PROB);
    for (int i = 0; i < 5; i++) {
        terrain.majorityRule();
    }
    population = new Population(p, plt, terrain);

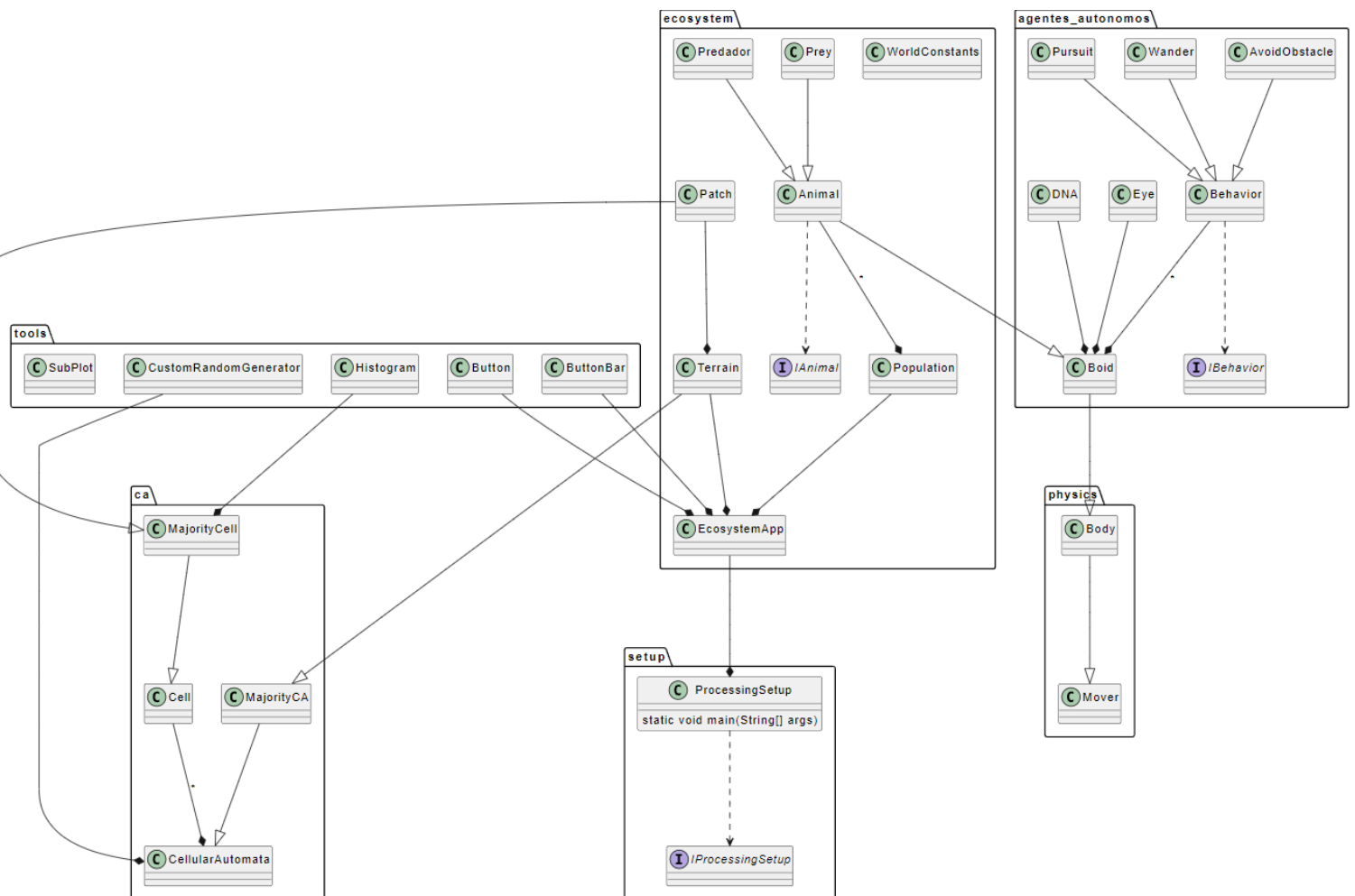
    //timer e update
    timer = 0;
    updateGraphTime = timer + intervalUpdate;
}
}

```

- Alterações na classe WorldConstants:

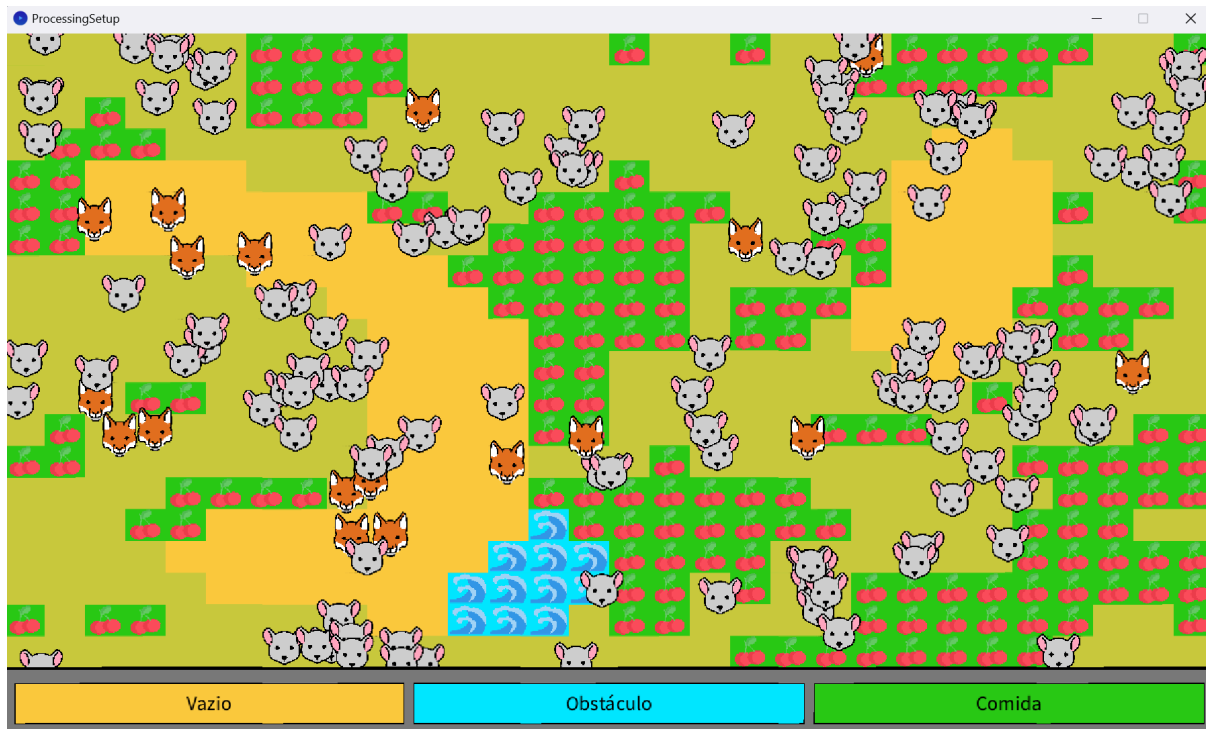
Os parâmetros de energia , número de colunas e de linhas foram alterados para garantir um equilíbrio do nosso ecossistema.

- UML



3. Resultados obtidos

Simulador interativo do nosso ecossistema:



4. Discussão de Resultados

Na nossa interface gráfica podemos observar o nosso ecossistema. As células referentes ao tipo de terreno com comida são inicializadas com a cor verde com uma imagem do fruto (cerejas). A azul estão as nossas células pertencentes ao tipo de terreno que consiste em obstáculos aos animais, como é o caso de lagos existentes no ambiente envolvente. Zonas desertas e zonas férteis ainda por acartar com alimento também são possíveis de visualizar a amarelo e verde claro, respetivamente.

A fim de criar uma interação pessoa-máquina, foram adicionados botões que permitem modificar o terreno existente no ecossistema, podendo ser adicionadas áreas rodeadas só com obstáculos, por exemplo, para isolar uma espécie.

O alimento presente é regenerado após 10 segundos.

5. Conclusão

Em suma, o projeto desenvolvido alcançou com sucesso o objetivo de implementar um simulador interativo que modela um ecossistema natural com presas, predadores e elementos ambientais diversificados. Durante o processo, foi possível abordar conceitos fundamentais de sistemas complexos e vida artificial, aplicando-os na simulação de dinâmicas ecológicas e no comportamento emergente das espécies representadas.

Os resultados mostram que o equilíbrio entre as populações de presas e predadores pode ser influenciado por fatores como disponibilidade de alimentos, obstáculos no ambiente e características comportamentais. Isso evidencia a relevância da interação entre espécies e o meio ambiente na dinâmica ecológica. A regeneração do alimento e a introdução de obstáculos também demonstraram como pequenas mudanças podem alterar significativamente a evolução do ecossistema.

6. Bibliografia

- Vídeos fornecidos pelo professor (plataforma Moodle);
- site oficial Processing em: <https://processing.org/>;
- site utilizado para obter as imagens necessários em: <https://openmoji.org/>.